



INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Asignatura:

Programación Avanzada

Proyecto 1

Sistema de Gestión de Productos e Inventario

Entrega 2 - Mantenimiento evolutivo

Profesor:

Ing. Juan Jose Vanzetti

Integrantes:

Albarracin, Trinidad	trinialbarracin13@gmail.com	15660
Magallanes, Agustin	agus57cr@gmail.com	16045
Mansilla, Santiago	santiagomansilla025@gmail.com	15921
Nardi, Elisa Maria	unielisanardi@gmail.com	16136
Olguin, Juan Cruz	olguinjuancruz@gmail.com	17241

Fecha de Entrega: 28/09/2026

Índice

5.1 Análisis de Dominio	3
Alcance del análisis	3
Impacto de los pedidos de cambio	3
Entidades, Value Objects y agregados	3
Cambios en el agregado Producto	5
Reglas de negocio identificadas	5
Decisiones de modelado	6
5.2 Event Storming	7
Reflexión sobre el Event Storming	8
5.3 Historias de Usuario	9
CR-001: Validación de datos	9
CR-002: Presentación del producto	9
CR-003: SuperLínea	9
H.U.3.1 – Asociar una Línea a una SuperLínea	10
H.U.3.2 – Gestionar el catálogo de SuperLíneas	10
CR-004: Búsqueda por Denominación, Línea, SuperLínea	11
H.U.4.1 – Buscar productos por denominación parcial	11
HU-004.2 – Buscar productos por nombre parcial de Línea	11
HU-004.3 – Buscar productos por nombre parcial de SuperLínea	12
HU-004.4 – Combinar los filtros de búsqueda	12
CR-005: Denominación automática	13
CR-006: Actualización masiva de precios	14
CR-007: Historial de precios	14
5.4 Desarrollo	16
Refactorización	16
Cambios y realización de CRs	17
CR-001 – Validación de Datos	17
Backend	17
Frontend	17
Testing	17
Resultado	17
CR-003 – Incorporación de SuperLíneas	18
Backend	18
Frontend	18
Testing	18
Resultado	18
CR-004 – Búsqueda de productos con coincidencias parciales	19

Frontend	19
Testing	19
Resultado	19
CR-006 – Actualización masiva de precios	20
Backend	20
Frontend	20
Resultado	20
CR-007 – Historial de precios	21
Backend	21
Frontend	21
Resultado	21
CR-002 y CR-005 – Presentaciones y Denominación Automática	22
Backend	22
Frontend	22
Correcciones realizadas durante la implementación	22
Resultado	22
Deuda técnica	23
5.5 Testing	25
Resumen de pruebas	25
Cambios hechos en el testing	25
5.6 Métricas y Gestión del Proyecto	27
Organización del trabajo	27
Plan de trabajo consolidado	27
Métricas registradas	29
Gestión de versiones e historial de Git	30
Retrospectiva del equipo	30
5.7 Trazabilidad	32

5.1 Análisis de Dominio

Alcance del análisis

El análisis se concentró en el contexto de gestión de productos e inventario y en el impacto de los pedidos de cambio CR-001 a CR-007.

El concepto central es Producto, ya que concentra las reglas relacionadas con denominación, costo, margen, precio, stock y clasificación. Los módulos heredados de usuarios, proveedores, clientes, documentos comerciales e importaciones se conservaron en el repositorio, pero no fueron modificados salvo cuando resultaron necesarios para relacionar un producto con otro concepto existente.

Impacto de los pedidos de cambio

CR	Impacto en el dominio
CR-001	Incorporó y reforzó invariantes para datos de Producto, especialmente denominación, costo, margen, precio y stock.
CR-002	Incorporó Presentación como entidad independiente y como relación opcional de Producto.
CR-003	Incorporó SuperLínea como nivel superior de clasificación y agregó la relación entre Línea y SuperLínea.
CR-004	No incorporó entidades nuevas; amplió la consulta de productos mediante coincidencias parciales por denominación, Línea y SuperLínea
CR-005	Incorporó la regla de generación automática de denominación a partir de Marca, Línea y Presentación, con posibilidad de edición manual.
CR-006	Incorporó la actualización masiva de precios mediante porcentaje o monto.
CR-007	Incorporó HistorialPrecio para conservar los cambios individuales de precio, costo y margen.

Entidades, Value Objects y agregados

Para identificar nuevas entidades, value objects, reglas de negocio y cambios a implementar lo primero que hicimos fue relevar lo que ya estaba presente en el programa y clasificarlo según si estaba o no presente en el dominio del mismo.

En primer lugar extrajimos las reglas de negocios presentes en el documento de análisis, para luego realizar un cuadro en el cual podamos observar las entidades importantes para el dominio y las que no con sus correspondientes justificaciones.

Concepto	Clasificación	Importante para el dominio	Justificación
Producto	Entidad y raíz de agregado	SI	Tiene identidad, ciclo de vida propio y concentra reglas de precio, stock y denominación.
Marca	Entidad	SI	Se gestiona de forma independiente y posee identidad propia.
Línea	Entidad	SI	Clasifica productos y se vincula con una SuperLínea. Posee identidad y ciclo de vida propios.
SuperLínea	Entidad	SI	Agrupar múltiples líneas con ciclo de vida independiente.
Presentación	Entidad	SI	Posee identidad, gestor propio y ciclo de vida independiente. Puede asociarse a distintos productos.
Historial Precio	Entidad	SI	Registra un cambio concreto de precio, con fecha, motivo, valores anteriores y nuevos, producto y usuario.
Movimiento Stock	Entidad	SI	Representa cada variación de stock de un producto y permite mantener trazabilidad de cantidad, motivo, tipo y fecha.
Costo	Value Object	SI	Representa un valor monetario no negativo.
Margen	Value Object	SI	Representa un porcentaje entre 0 y 1.
Precio	Value Object	SI	Se deriva de costo y margen, y debe ser mayor que cero.

Stock	Value Object	SI	Representa la cantidad disponible y no admite valores negativos.
-------	--------------	----	--

Cambios en el agregado Producto

Los cambios principales sobre Producto fueron:

- Incorporación de la relación con Presentación.
- Incorporación del indicador denominacionEditadaManualmente.
- Generación automática de denominación con la regla: Marca + Línea + Presentación.
- Incorporación de los Value Objects Costo, Margen, Precio y Stock.
- Incorporación de operaciones de actualización de precio y ajuste de stock.
- Incorporación de la relación con HistorialPrecio para registrar los cambios individuales de precio.
- Incorporación de filtros por denominación de Línea y SuperLínea.

Reglas de negocio identificadas

- ❖ El precio se calcula mediante la fórmula: $\text{Precio} = \text{Costo} \times (1 + \text{Margen})$.
- ❖ El margen general es del 15%, pero algunos productos pueden definir un margen especial. Es el Producto quien conoce su propio costo y margen.
- ❖ El stock de producto debe ser mayor al de stock crítico. En caso de ser igual o menor se debe alertar.
- ❖ El stock nunca puede ser negativo.
- ❖ Permite modificar manualmente el stock de un producto, siempre indicando un motivo.
- ❖ Siempre se debe registrar el motivo por el cual se disminuye el stock de forma manual.
- ❖ Los movimientos y el stock actual deben mantenerse constantes entre sí.
- ❖ Un ajuste de stock requiere indicar un motivo y no puede generar stock negativo.
- ❖ La denominación no puede ser vacía
- ❖ Si no se informa una denominación manual, se genera con Marca, Línea y Presentación.
- ❖ Si el usuario edita la denominación manualmente, las modificaciones posteriores de Marca, Línea o Presentación no deben reemplazarla.

Decisiones de modelado

- Se modeló a Producto como raíz del agregado porque concentra las reglas de negocio relacionadas con precio, costo, margen, stock y denominación.
- Presentación se clasificó como entidad y no como Value Object, porque posee identidad propia, puede administrarse de manera independiente y puede asociarse a distintos productos.
- SuperLínea se incorporó como entidad para representar un nivel superior de clasificación de las Líneas. Cada Línea puede vincularse con una SuperLínea, permitiendo organizar el catálogo de productos.
- HistorialPrecio se modeló como entidad porque representa un registro individual, identificable y persistente de cada cambio de precio. Conserva el estado anterior y nuevo del precio, costo y margen, junto con la fecha, el motivo y el usuario responsable.
- Costo, Margen, Precio y Stock se modelaron como Value Objects porque se definen por su valor y encapsulan validaciones propias. Esta decisión permite evitar valores monetarios, porcentajes o cantidades inválidas dentro de Producto.
- MovimientoStock se consideró una entidad del dominio porque representa un hecho trazable sobre las variaciones de inventario y se vincula con Producto.

Reflexión sobre el Event Storming

Los eventos más relevantes identificados fueron la creación y actualización de un producto, el registro de una Presentación, la asociación de una Línea a una SuperLínea, la generación automática de la denominación, la actualización de precios y el registro del historial de precios.

Estos eventos permitieron identificar reglas de negocio que debían quedar explícitas en el modelo. Entre ellas se encuentran la generación de la denominación a partir de Marca, Línea y Presentación, la conservación de la denominación cuando fue editada manualmente, la validación de valores de costo, margen, precio y stock, y el registro de los cambios individuales de precio con su motivo y responsable.

El Event Storming también permitió advertir que los eventos vinculados al stock deben formar parte del dominio: un ajuste de stock y una situación de stock bajo son hechos relevantes para el negocio. Estos eventos se contemplan en el análisis, aunque su implementación completa queda registrada como deuda técnica.

La búsqueda parcial de productos no se consideró un evento de dominio, ya que representa una consulta sobre información existente y no un cambio en el estado del negocio.

Como resultado, el modelo se amplió con las entidades Presentación, SuperLínea e HistorialPrecio. Además, se consolidó a Producto como agregado principal y se incorporaron los Value Objects Costo, Margen, Precio y Stock para encapsular sus reglas de validación y cálculo.

5.3 Historias de Usuario

CR-001: Validación de datos

H.U.1

Como usuario, quiero que el sistema rechace valores de costo, precio o stock inválidos al cargar o editar un producto, para no terminar con datos incoherentes en el catálogo.

Criterios de Aceptación:

- **Given:** estoy creando o editando un producto,
When: ingreso un costo negativo,
Then: el sistema rechaza la operación y muestra un mensaje claro indicando que el costo no puede ser negativo.
- **Given:** estoy creando o editando un producto,
When: ingreso un stock o stock mínimo negativo,
Then: el sistema rechaza la operación con un mensaje específico para ese campo.
- **Given:** intento guardar un producto con datos válidos,
When: confirmo la operación,
Then: el producto se guarda sin errores.

CR-002: Presentación del producto

H.U.2

Como usuario, quiero poder asignarle una presentación (por ejemplo "1L" o "Pack x6") a un producto, para diferenciar variantes del mismo producto que se venden de formas distintas.

Criterios de Aceptación:

- **Given:** estoy en el formulario de producto,
When: seleccionó una presentación existente,
Then: el producto queda asociado a esa presentación.
- **Given:** necesito una presentación que no existe todavía,
When: la doy de alta desde el gestor de presentaciones,
Then: queda disponible para asociarse a cualquier producto.
- **Given:** una presentación está asociada a productos activos,
When: intento eliminarla,
Then: el sistema lo impide y explica por qué.

CR-003: SuperLínea

H.U.3.1 – Asociar una Línea a una SuperLínea

Como usuario, quiero que cada Línea pertenezca a una SuperLínea, para agrupar categorías relacionadas bajo un nivel superior de clasificación.

Criterios de Aceptación:

- **Given:** estoy creando una Línea y existen SuperLíneas activas,
When: consulto las opciones para asignarla ,
Then: el sistema muestra las SuperLíneas activas disponibles
- **Given:** Completé los demás datos válidos de una Línea.
When: Selecciono una SuperLínea activa y guardo.
Then: La Línea se guarda asociada a esa SuperLínea.
- **Given:** Estoy creando una Línea,
When: Intento guardarla sin SuperLínea,
Then: Se muestra “Debe seleccionar una SuperLínea para la línea” y no se guarda.
- **Given:** Estoy editando una Línea existente.
When: Cambio su SuperLínea por otra activa y guardo.
Then: La asociación se actualiza y se conserva al volver a consultar la Línea.
- **Given:** Estoy editando una Línea existente.
When: Intento dejarla sin SuperLínea y guardar.
Then: Se muestra el mensaje de obligatoriedad y se conserva la asociación anterior.
- **Given:** La SuperLínea enviada no existe o está dada de baja.
When: Intento guardar la asociación.
Then: El sistema rechaza la operación e informa que debe seleccionarse una SuperLínea válida.

H.U.3.2 – Gestionar el catálogo de SuperLíneas

Como usuario, quiero crear, modificar y dar de baja SuperLíneas, para mantener la clasificación utilizada por las Líneas.

Criterios de Aceptación:

- **Given:** Estoy creando una SuperLínea
When: Completo una denominación válida y guardo
Then: Se crea y queda disponible para asignar a Líneas.
- **Given:** Estoy creando o modificando una SuperLínea.
When: Intento guardar su denominación vacía o compuesta únicamente por espacios.
Then: El sistema informa que la denominación es obligatoria y no guarda los cambios.

- **Given:** Existe una SuperLínea activa.
When: Modifico su denominación por otra válida y guardo.
Then: Se actualiza conservando su identidad y las asociaciones de sus Líneas.
- **Given:** Una SuperLínea tiene al menos una Línea activa asociada,
When: intento darla de baja
Then: El sistema lo impide e informa que tiene líneas activas asociadas.
- **Given:** Una SuperLínea no tiene líneas activas asociadas,
When: solicito su baja y cumpla las demás condiciones de la operación,
Then: el sistema la da de baja lógicamente.

CR-004: Búsqueda por Denominación, Línea, SuperLínea

H.U.4.1 – Buscar productos por denominación parcial

Como usuario, quiero buscar productos escribiendo parte de su denominación, para encontrarlos sin recordar su nombre completo.

Criterios de Aceptación:

- **Given:** existen los productos “Coca-Cola 2L”, “Coca-Cola 1L” y “Sprite 2L”.
When: ingreso “coca” en el filtro de denominación y ejecuto la búsqueda.
Then: se devuelven “Coca-Cola 2L” y “Coca-Cola 1L”, y no se devuelve “Sprite 2L”.
- **Given:** existe el producto “Coca-Cola 2L”.
When: ingreso “cola” en el filtro de denominación y ejecuto la búsqueda.
Then: el producto aparece, porque el texto puede coincidir en cualquier parte de la denominación.

HU-004.2 – Buscar productos por nombre parcial de Línea

Como usuario autorizado, quiero buscar productos escribiendo parte del nombre de su Línea, para encontrar productos de categorías relacionadas sin escribir sus nombres completos.

Criterios de aceptación:

- **Given:** existen productos pertenecientes a las Líneas “Gaseosas”, “Aguas Minerales” y “Jugos Naturales”.
When: ingreso “gas” en el filtro de Línea y ejecuto la búsqueda.
Then: se devuelven los productos pertenecientes a “Gaseosas” y se excluyen los de las otras dos Líneas.
- **Given:** existen productos pertenecientes a las Líneas “Jugos Naturales” y “Jugos Concentrados”.
When: ingreso “jugos” en el filtro de Línea y ejecuto la búsqueda.
Then: se devuelven los productos de ambas Líneas.

HU-004.3 – Buscar productos por nombre parcial de SuperLínea

Como usuario autorizado, quiero buscar productos escribiendo parte del nombre de su SuperLínea, para consultar productos de un nivel superior de clasificación.

Criterios de aceptación:

- **Given:** la SuperLínea “Bebidas Sin Alcohol” agrupa las Líneas “Gaseosas” y “Jugos”, y ambas tienen productos.
When: ingreso “beb” en el filtro de SuperLínea y ejecuto la búsqueda.
Then: se devuelven los productos de ambas Líneas.
- **Given:** existen las SuperLíneas “Bebidas Sin Alcohol”, “Bebidas Alcohólicas” y “Alimentos”, con productos en sus respectivas Líneas.
When: ingreso “bebidas” en el filtro de SuperLínea y ejecuto la búsqueda.
Then: se devuelven los productos de las dos SuperLíneas coincidentes y se excluyen los pertenecientes a “Alimentos”.

HU-004.4 – Combinar los filtros de búsqueda

Como usuario autorizado, quiero combinar los filtros de denominación, Línea y SuperLínea, para obtener resultados más específicos.

Criterios de aceptación:

- **Given:** existen productos de distintas Líneas y SuperLíneas.
When: ingreso “cola” en denominación y “beb” en SuperLínea y ejecuto la búsqueda.
Then: se devuelven únicamente los productos cuya denominación contiene “cola” y cuya SuperLínea contiene “beb”.
- **Given:** existen productos de distintas clasificaciones.
When: ingreso “cola” en denominación, “gas” en Línea y “sin alcohol” en SuperLínea.
Then: se devuelven únicamente los productos que cumplen los tres criterios simultáneamente.
- **Given:** ingreso filtros de Línea y SuperLínea que no coinciden para ningún producto.
When: ejecuto la búsqueda.
Then: no se devuelven productos; el sistema no ignora ninguno de los filtros para obtener resultados.

Criterios de aceptación comunes de CR-004:

Los siguientes criterios se aplican a las historias HU-004.1, HU-004.2, HU-004.3 y HU-004.4, según corresponda. Se presentan una sola vez para evitar duplicaciones.

CA-C01 – Mayúsculas y minúsculas

- **Given:** existen productos que coinciden con un texto de búsqueda.

- **When:** ingreso ese texto con distintas combinaciones de mayúsculas y minúsculas.
- **Then:** el sistema devuelve los mismos resultados.

CA-C02 – Espacios al principio y al final

- **Given:** ingreso un texto con espacios al principio o al final.
- **When:** ejecuto la búsqueda.
- **Then:** el sistema ignora esos espacios exteriores al buscar coincidencias.

CA-C03 – Filtros vacíos

- **Given:** completo algunos filtros y dejo otros vacíos o únicamente con espacios.
- **When:** ejecuto la búsqueda.
- **Then:** se aplican solamente los filtros que contienen texto.

CA-C04 – Búsqueda sin resultados

- **Given:** no existen productos que cumplan los filtros ingresados.
- **When:** ejecuto la búsqueda.
- **Then:** el sistema muestra una lista vacía y el mensaje “No se encontraron productos con los filtros ingresados”, sin presentarlo como un error.

CA-C05 – Limpiar filtros

- **Given:** hay filtros de denominación, Línea o SuperLínea aplicados.
- **When:** limpio los tres filtros y ejecuto la búsqueda.
- **Then:** se muestra el listado sin restricciones por esos campos, respetando los demás filtros y permisos existentes.

CA-C06 – Conservar filtros al cambiar de página

- **Given:** los resultados de la búsqueda ocupan varias páginas.
- **When:** cambio de página.
- **Then:** se mantienen los filtros y el total corresponde a los productos que coinciden con ellos.

CA-C07 – Reiniciar la paginación

- **Given:** estoy en una página posterior a la primera.
- **When:** cambio los filtros y ejecuto una nueva búsqueda.
- **Then:** el sistema muestra la primera página de los nuevos resultados.

CR-005: Denominación automática

H.U.5

Como usuario, quiero que la denominación del producto se genere automáticamente a partir de Marca + Línea + Presentación, para no tener que escribirla a mano cada vez y evitar inconsistencias de formato.

Criterios de Aceptación:

- **Given:** completé Marca, Línea y Presentación de un producto nuevo,
When: no edito manualmente el campo denominación,
Then: el sistema genera la denominación combinando esos tres valores (por ejemplo, "Coca-Cola Gaseosas 2L").
- **Given:** la denominación autogenerada no me sirve tal cual,
When: la edito manualmente,
Then: el sistema respeta el valor editado y no la vuelve a sobrescribir automáticamente.
- **Given:** cambio la Marca, Línea o Presentación de un producto ya creado,
When: la denominación no fue editada manualmente,
Then: el sistema genera nuevamente la denominación, en base a los nuevos valores.
- **Given:** falta alguno de los tres componentes,
When: se intenta generar la denominación automática,
Then: el sistema solicita se ingrese los componentes faltantes.

CR-006: Actualización masiva de precios

H.U.6

Como usuario, quiero actualizar el precio de muchos productos a la vez por porcentaje o por monto fijo, filtrando por línea o aplicando a todo el catálogo, para no tener que editar producto por producto ante un ajuste general.

Criterios de Aceptación:

- **Given:** quiero aumentar precios un 10%,
When: aplico el ajuste por porcentaje a una Línea específica,
Then: todos los productos de esa Línea recalculan su precio con ese porcentaje aplicado sobre el valor actual.
- **Given:** quiero aplicar un ajuste por monto fijo,
When: indico el monto y el alcance (sea Línea o global),
Then: el sistema suma o resta ese monto al precio de cada producto alcanzado.
- **Given:** el ajuste genera un precio negativo en algún producto,
When: se ejecuta la actualización masiva,
Then: el sistema rechaza la operación completa.
- **Given:** se ejecutó una actualización masiva,
When: reviso el historial de precios de cualquiera de los productos afectados,
Then: cada cambio queda registrado individualmente (relacionado con CR-007).

CR-007: Historial de precios

H.U.7

Como usuario, quiero que cada cambio de precio de un producto quede registrado con precio anterior, precio nuevo, fecha y motivo, para poder auditar por qué y cuándo cambió el precio de cualquier producto.

Criterios de Aceptación:

- **Given:** cambio el precio de un producto,
When: confirmo el cambio,
Then: se crea un registro de historial con precio anterior, precio nuevo, fecha y motivo.
- **Given:** intento registrar un cambio de precio,
When: el nuevo precio es menor o igual a 0,
Then: el sistema rechaza el cambio.
- **Given:** quiero consultar el historial de un producto,
When: accedo a su detalle,
Then: puedo ver la lista de cambios de precio ordenados cronológicamente.
- **Given:** se actualiza el precio sin indicar motivo,
When: se intenta guardar,
Then: el sistema exige que se ingrese un motivo.

5.4 Desarrollo

Refactorización

La implementación presenta un comienzo de separación entre capas de dominio, aplicación e infraestructura.

- ❖ El dominio contiene a las entidades del negocio, los value-objects.
- ❖ La aplicación se ocupa de lógica de negocio pura y validaciones de negocio.
- ❖ La infraestructura se ocupa de la persistencia con la base de datos (parte de más bajo nivel del sistema) y la comunicación con el exterior (controllers con protocolo HTTP por ejemplo).

El problema de la implementación actual, es la dependencia del ORM elegido en capas que no son de infraestructura. Por ejemplo, los servicios manipulan objetos del tipo Product, siendo este una entidad con TypeORM implementado. Es decir, la capa de aplicación conoce a la parte de persistencia de la capa de infraestructura, lo cual no es correcto en términos de separación de capas, y puede llevar a la refactorización de múltiples capas a la hora de migrar a otro ORM, por ejemplo, Prisma.

La solución es concretar la separación de la siguiente manera:

domain/

entities/	(carentes de relación con el ORM)
value-objects/	
factories/	(para construcción de objetos con value objects y cierta lógica)
inputs/	(interfaces de input para factories)
services/	(servicios de dominio)
repositories/	(interfaces de repositorio)
exceptions/	(excepciones de dominio, como InvalidPriceException)

application/

services/	(servicio o fachada que agrupe los use cases)
use-cases/	(casos de uso con lógica atómica de negocio)
dtos/	(de request y response, con validaciones acorde a las reglas de negocio)
mappers/	(como transformar dtos en entidades y viceversa: toDomain(), toResponse())
validators/	

infraestructura/

presentacion/	
controllers/	
interceptors/	
exceptions/	(transformar excepciones de dominio en, en este caso, códigos HTTP)
persistence/	
entities/	
mappers/	(como toDomain(), toOrm())
repositories/	(implementaciones de las interfaces de repos del dominio)

Con esta estructura, se pueden integrar de forma correcta los value-objects encontrados a las entidades, centralizar las validaciones de dominio en estos y en las entidades, permitir que la aplicación se ocupe de la lógica de negocio sin encargarse de reglas de dominio y persistencia, y que la infraestructura se ocupe de los extremos de la aplicación, comunicación y traducción del exterior, y comunicación con la base de datos.

Cambios y realización de CRs

CR-001 – Validación de Datos

Rama: feature/cr-001

Backend

- Se reforzaron las validaciones de negocio en Producto, Marca y Línea para impedir el ingreso de datos inválidos.
- Se incorporaron excepciones de dominio específicas para informar errores de validación de forma clara y consistente.
- Se validó que los campos de denominación sean obligatorios y no acepten valores vacíos o compuestos únicamente por espacios.
- Se ajustaron los Value Objects de Costo y Precio: el costo no admite valores negativos y el precio resultante debe ser mayor que cero.
- Se implementó la validación obligatoria del motivo en las operaciones de ajuste de stock.
- Se agregó la validación de stock mínimo mayor o igual a cero en la entidad Línea.
- Se actualizaron los DTOs de creación y edición incorporando validaciones mediante class-validator para costos, stock y stock mínimo.

Frontend

- Se actualizaron los esquemas de validación de formularios utilizando Yup.
- Se incorporó validación para evitar denominaciones vacías en productos, marcas y líneas.
- Se alinearon las reglas de validación numérica con las reglas definidas en Backend.
- Se mejoraron y unificaron los mensajes de error mostrados al usuario.

Testing

- Se ampliaron y actualizaron los tests unitarios para validar los nuevos escenarios de error y prevenir regresiones.
- Se verificó el correcto funcionamiento de las validaciones implementadas en entidades, Value Objects y formularios.

Resultado

- El sistema impide registrar productos, marcas y líneas con datos inconsistentes.
- Los precios y costos inválidos son rechazados antes de ser persistidos.
- Los usuarios reciben mensajes de error claros y específicos.
- Se garantiza una mayor integridad de los datos tanto en frontend como en backend.

CR-003 – Incorporación de SuperLíneas

Rama: feature-cr3

Backend

- Se incorporó la entidad SuperLínea como nivel superior de clasificación de las Líneas.
- Se creó el módulo de SuperLínea con entidad de dominio, DTOs, casos de uso, contrato de repositorio, persistencia, mappers, entidad ORM y controlador.
- Se implementaron las operaciones de alta, consulta, modificación y baja lógica.
- La denominación de SuperLínea es obligatoria, no admite espacios como único contenido y posee un máximo de 255 caracteres.
- Se agregó la relación entre Línea y SuperLínea mediante superlineald.
- Al crear o modificar una Línea se valida que la SuperLínea seleccionada exista y se encuentre activa.
- La baja de una SuperLínea se rechaza cuando posee Líneas activas asociadas.
- Se incorporó una clave foránea para preservar la integridad de la relación entre Línea y SuperLínea.
- Se creó una migración para generar la tabla de SuperLíneas, relacionarla con Línea y asociar las Líneas existentes a una SuperLínea inicial denominada "General".
- Se actualizaron los datos iniciales de Líneas para incluir su SuperLínea asociada.

Frontend

- Se incorporó el catálogo de SuperLíneas con opciones de alta, edición, consulta y baja.
- En el formulario de Línea se agregó un selector obligatorio de SuperLíneas activas.
- Se incorporó validación mediante Yup para impedir guardar una Línea sin SuperLínea.
- Al editar una Línea, el formulario recupera y muestra la SuperLínea previamente asociada.
- La SuperLínea asociada se visualiza en el listado, tarjetas y ventana de información de Línea.

Testing

- Se incorporaron pruebas sobre la creación, actualización y baja de SuperLíneas.
- Se contemplaron validaciones de denominación, asociación obligatoria de Línea y restricción de baja cuando existen Líneas activas asociadas.

Resultado

Se incorporó una clasificación superior para las Líneas, permitiendo organizar el catálogo de productos mediante la relación Producto → Línea → SuperLínea. La relación se mantiene en frontend, backend y base de datos.

CR-004 – Búsqueda de productos con coincidencias parciales

Rama: feature-cr4

Backend

- Se amplió la búsqueda de productos para admitir coincidencias parciales por denominación, nombre de Línea y nombre de SuperLínea.
- Se incorporaron los parámetros opcionales `lineaDenominacion` y `superlineaDenominacion` al DTO de búsqueda.
- Los filtros recorren las capas de controlador, servicio, caso de uso, contrato de repositorio y repositorio.
- La búsqueda por SuperLínea utiliza la relación Producto → Línea → SuperLínea, sin incorporar una relación directa entre Producto y SuperLínea.
- Se creó `producto-search.helper.ts` para centralizar la construcción de filtros parciales.
- Los textos ingresados se normalizan eliminando espacios externos; los campos vacíos no incorporan condiciones a la consulta.
- La consulta utiliza LIKE, comparación sin distinción entre mayúsculas y minúsculas, parámetros de consulta y escape de los caracteres % y _.
- Cuando se utilizan varios criterios, se combinan mediante AND.
- La búsqueda se aplica antes de la paginación, excluye productos dados de baja lógica y mantiene un orden estable por denominación e identificador.

Frontend

- Se incorporó un componente con filtros de texto para Denominación, Línea y SuperLínea.
- Se mantuvo la búsqueda rápida existente por código.
- Los filtros se aplican al presionar el botón Buscar o la tecla Enter.
- Se separaron los valores ingresados de los filtros aplicados, evitando modificar resultados mientras el usuario escribe.
- Al ejecutar una nueva búsqueda, la paginación vuelve a la primera página y conserva los filtros al cambiar de página.
- Se agregó un resumen de los filtros aplicados.
- El botón Limpiar elimina los criterios de texto y vuelve a consultar los productos.
- Cuando no existen coincidencias, se muestra el mensaje: “No se encontraron productos con los filtros ingresados”.

Testing

- Se incorporaron pruebas para filtros por denominación, Línea y SuperLínea.
- Se contemplaron búsquedas parciales, combinación de filtros, campos vacíos, mayúsculas y minúsculas, paginación y productos dados de baja.

Resultado

El sistema permite localizar productos sin conocer su denominación completa y combinar criterios de búsqueda por producto, Línea y SuperLínea. La funcionalidad conserva la paginación y diferencia la ausencia de resultados de los errores de comunicación.

CR-006 – Actualización masiva de precios

Rama: feature/CR-006

Backend

- Se implementó la funcionalidad de actualización masiva de precios mediante porcentaje o monto fijo.
- Se creó el DTO de la operación incorporando validaciones según el tipo de actualización.
- Se desarrolló la lógica de previsualización para calcular los nuevos precios antes de aplicar los cambios.
- Se implementó la validación integral del lote para garantizar que todos los productos sean válidos antes de persistir la operación.
- Se agregaron los endpoints para:
 - Previsualizar cambios masivos.
 - Confirmar y guardar la actualización masiva.
- Se adaptó el modelo de dominio para soportar actualizaciones directas de precios, registrando usuario y fecha de modificación.

Frontend

- Se implementó el flujo completo de actualización masiva de precios.
- Se incorporó la búsqueda y filtrado de productos por marca, línea y sublínea.
- Se agregó la posibilidad de aplicar cambios mediante:
 - Porcentaje.
 - Monto fijo.
- Se habilitó el uso de valores negativos únicamente para actualizaciones por monto fijo.
- Se desarrolló la vista previa mostrando precios actuales y precios resultantes antes de confirmar la operación.
- Se integraron los servicios necesarios para consumir los endpoints de previsualización y guardado.

Resultado

- Los usuarios pueden actualizar precios de forma masiva sobre un conjunto de productos o una línea específica.
- El sistema permite previsualizar el impacto de los cambios antes de aplicarlos.
- Se garantiza que la operación sea validada completamente antes de persistir modificaciones.
- Se reducen tiempos operativos en tareas de ajuste general de precios.

CR-007 – Historial de precios

Rama: feature/CR-007

Backend

- Se creó la entidad HistorialPrecio con los atributos necesarios para registrar cambios de precio, costo y margen de un producto.
- Se implementaron los types, factory, mapper, interfaz y repositorio para la gestión y persistencia del historial.
- Se creó el caso de uso FindHistorialPrecioUseCase para consultar el historial de cambios de un producto.
- Se incorporó el DTO y mapper correspondiente para la respuesta del historial de precios.
- Se generó la migración para crear la tabla de historial de precios.
- Se modificó la actualización de precios para:
 - Registrar automáticamente el estado anterior y nuevo del producto.
 - Guardar el motivo del cambio.
 - Validar que el costo ingresado sea mayor a cero.
- Se agregaron los endpoints:
 - PATCH /:id/precio
 - GET /:id/historial-precio
- Se registraron las nuevas dependencias y repositorios en el módulo de productos.

Frontend

- Se creó la interfaz HistorialPrecio.
- Se incorporó el campo obligatorio motivo en el formulario de cambio de precio.
- Se implementó el modal de actualización de precios con:
 - Costo.
 - Margen.
 - Motivo.
 - Vista previa del precio calculado.
- Se desarrolló el hook useHistorialPrecio para obtener y administrar el historial.
- Se creó la tabla de historial de precios mostrando:
 - Valores anteriores y nuevos.
 - Motivo del cambio.
 - Usuario responsable.
 - Fecha de modificación.
- Se agregaron los métodos necesarios en el servicio de productos para actualizar precios y consultar el historial.
- Se incorporaron validaciones para garantizar que el costo sea mayor a cero y que el motivo sea obligatorio.

Resultado

- Cada modificación de precio queda registrada con su motivo, valores anteriores y nuevos, fecha y usuario responsable.
- Los usuarios pueden consultar el historial completo de cambios de precio de un producto.

CR-002 y CR-005 – Presentaciones y Denominación Automática

Rama: feature/CR-002-y-CR-005

Backend

- Se implementó el nuevo módulo **Presentación** con funcionalidades de alta, edición, consulta y baja.
- Se incorporó la relación entre **Producto** y **Presentación**.
- Se implementó la restricción que impide eliminar una presentación asociada a productos activos.
- Se agregó un endpoint para obtener presentaciones desde el formulario de productos.
- Se desarrolló la generación automática de la denominación del producto a partir de **Marca + Línea + Presentación**.
- Se incorporó la lógica que permite mantener una denominación editada manualmente sin que sea sobrescrita por actualizaciones posteriores.
- Se agregaron validaciones de dominio para garantizar que la denominación automática solo pueda generarse cuando exista una presentación asociada.
- Se creó la migración necesaria para incorporar la entidad Presentación y los nuevos campos asociados en Producto.

Frontend

- Se desarrolló la pantalla de gestión de Presentaciones siguiendo el mismo patrón utilizado en Marca y Línea.
- Se integró el selector de Presentación dentro del formulario de productos.
- Se habilitó la carga automática de la presentación asociada al editar un producto.
- Se modificó el campo Denominación para permitir generación automática o edición manual.
- Se actualizaron las interfaces y modelos de datos para alinearlos con la estructura expuesta por el backend.

Correcciones realizadas durante la implementación

- Se corrigió un error que impedía crear y editar productos debido al envío de campos obsoletos al backend.
- Se eliminó del módulo de productos la lógica asociada a IVA y cotización en dólares, ya que no forman parte del dominio actual.
- Se corrigió la carga inicial de los desplegables de Marca, Línea y Presentación para que muestren información al abrir el formulario sin requerir búsquedas manuales.

Resultado

- Los productos pueden asociarse a una Presentación administrable desde su propio gestor.
- La denominación puede generarse automáticamente a partir de Marca, Línea y Presentación.
- Los usuarios pueden sobrescribir la denominación manualmente cuando sea necesario.
- Se garantiza consistencia en la información mostrada y almacenada en los productos.
- Se mejoró la experiencia de creación y edición de productos mediante correcciones funcionales y de integración.

Deuda técnica

Durante el desarrollo se priorizaron los pedidos de cambio CR-001 a CR-007 y la refactorización del módulo de productos. Las funcionalidades heredadas que no forman parte de ese alcance, junto con las inconsistencias detectadas en la revisión de la Entrega 3, se registran como deuda técnica para una evolución posterior.

Área	Estado actual y pendiente
Movimiento de stock	El módulo MovimientoStock mantiene una estructura parcial. La entidad no registra cantidad ni motivo, y el servicio asociado conserva métodos generados sin lógica funcional. Los ajustes de stock actualizan el producto, pero no generan un movimiento persistido que permita conocer qué cambió, en qué cantidad y por qué motivo.
Alertas de stock	La entidad Producto incorpora el método estaBajoMinimo(), pero su resultado no se expone al frontend ni se muestra como alerta visual en el catálogo. Además, el análisis define alerta cuando el stock es menor o igual al mínimo, mientras que el método actual utiliza solamente la condición menor que.
Eventos de dominio de stock	Los eventos StockBajo y StockActualizado, definidos en el análisis, no se encuentran implementados. Por lo tanto, el sistema no cuenta con un mecanismo para reaccionar automáticamente ante un ajuste de stock o una situación de stock crítico.
Actualización masiva de precios	El cambio masivo actualiza directamente el precio de los productos, pero no registra un historial por cada producto actualizado ni utiliza la misma transacción que la actualización individual de precios. Debe unificarse este flujo con las reglas de costo, margen, precio e historial definidas para el dominio.
Usuarios y roles	El sistema posee autenticación, usuarios y roles, pero este subdominio no está incorporado de forma completa al análisis de dominio. También existen nombres de roles no uniformes entre distintas partes del código, como Root, Administrador y Admin.
Registro y seguridad de usuarios	El registro de usuarios existe en el backend, pero requiere revisión funcional y de seguridad. Quedan pendientes la validación del estado activo del usuario al iniciar sesión, la revisión del flujo de recuperación de contraseña y la definición de políticas de sesión, renovación y revocación de tokens.

Proveedor y datos organizativos	El código base incluye módulos de Proveedor, Cliente, Personal, Empresa, domicilio y datos impositivos. Estos conceptos no fueron desarrollados en el análisis del dominio de productos e inventario y requieren un análisis propio de sus reglas, relaciones y validaciones.
Operaciones comerciales	Las compras, ventas, devoluciones y documentos comerciales permanecen fuera del alcance. Como evolución futura, deben vincularse con Producto y MovimientoStock para asegurar que cada operación comercial actualice y deje trazabilidad sobre el stock.
Importaciones y listas de precios	El código heredado contiene funcionalidades relacionadas con importación, comparación y listas de precios. Queda pendiente definir su relación con el precio calculado por costo y margen, validar los datos importados y registrar su origen dentro del historial de precios.
Notificaciones y auditoría	Se encuentran pendientes las notificaciones persistentes de stock bajo y una auditoría unificada de operaciones relevantes, como cambios de precio, ajustes de stock y modificaciones de entidades.

Decisión tomada: Estas funcionalidades se conservaron en el repositorio, pero no se modificaron durante esta entrega porque los CR asignados se concentraron en la evolución del catálogo de productos, Presentación, SuperLínea, búsquedas, denominación, precios e historial de precios. Su implementación completa requiere ampliar el análisis de dominio y definir reglas de negocio que exceden el alcance de los cambios solicitados.

5.5 Testing

Resumen de pruebas

Módulo / área	Capa	Herramienta	Suites / archivos de test	Tests ejecutados	Resultado
Producto + Movimiento Stock	Backend	Jest	46 archivos (8 specs previos)	-	Tests implementados y ejecutados
Producto + Superlínea	Backend / Frontend	Jest / Vitest	43 archivos (5 specs previos)	-	Tests implementados y ejecutados
Producto + Precios	Frontend	Vitest + RTL + jsdom	34 suites	114	Exitosos
Línea + Presentación	Backend / Frontend	Jest + Vitest + Supertest	14 suites	62	Exitosos
Marca + Presentación	Backend / Frontend	Jest + Vitest	31 nuevos + 7 previos	-	Cobertura ampliada
Búsqueda por denominación, Línea y SuperLínea	Backend / Frontend	Jest / Vitest + RTL	2 suites	20	Exitosos

Cambios hechos en el testing

Módulo	Área testeada	Principales escenarios	Resultado / comportamiento verificado
Producto – Backend	Application	DTOs, mappers, servicios y casos de uso	Validación de datos, transformación y ejecución correcta de operaciones
Producto – Backend	Domain	Entidad, excepciones, servicios y Value Objects	Validación de reglas de negocio e invariantes
Producto – Backend	Infraestructure	ORM, mappers, repositorios y validadores	Correcto mapeo y persistencia; validaciones de unicidad y relaciones

Producto – Frontend	Producto	Formatos, validaciones, cálculo de precios, búsqueda y paginación	Casos válidos, inválidos, errores de servicio y comportamiento de controles
Producto – Búsqueda	Backend / Frontend	Denominación, Línea, SuperLínea, filtros combinados y paginación	Coincidencias parciales, filtros AND, limpieza, conservación y reinicio de paginación
Precios – Frontend	Cambio de precios	Cambio masivo, edición manual, filtros y servicios	Cálculo, guardado, actualización local, validaciones y manejo de errores
Importaciones – Frontend	IVECO / Nex-Pro	Formularios, archivos, cotizaciones y comparación	Validaciones y flujos de importación
Movimiento Stock	Backend	Entidad, controller, service y mapper	Correcto funcionamiento de las operaciones relacionadas con stock
Superlínea	Backend / Frontend	Gestión y componentes	Validación del comportamiento del módulo
Línea	Backend	Application, Domain, Infrastructure	CRUD, reglas de dominio, SuperLínea válida, unicidad y baja lógica
Línea	Frontend	Formularios, filtros y servicios	Validaciones, transformación de datos y acciones del usuario
Presentación	Backend / Frontend	DTOs, entidad, casos de uso, repositorio y UI	Validaciones, transformación, persistencia y manejo HTTP
Marca	Backend / Frontend	DTOs, casos de uso, dominio, infraestructura y componentes	Validaciones, reglas de eliminación, persistencia y comportamiento de UI

5.6 Métricas y Gestión del Proyecto

Organización del trabajo

El equipo estuvo integrado por cinco participantes. La organización del desarrollo se realizó mediante la distribución de pedidos de cambio y el uso de Git y GitHub para registrar implementaciones, correcciones, pruebas e integraciones.

Se utilizaron ramas específicas para desarrollar funcionalidades y ramas de integración para reunir los aportes del equipo. Posteriormente, el trabajo de testing se distribuyó por módulos y capas, procurando que las responsabilidades estuvieran identificadas.

No se utilizó una herramienta independiente de gestión de tareas durante el desarrollo. El siguiente cuadro consolida el plan y las actividades realizadas, tomando como referencia las ramas, los commits y la documentación de la entrega.

Plan de trabajo consolidado

Actividad / CR	Trabajo realizado	Responsable	Evidencia
Análisis del sistema inicial	Revisión del código heredado, identificación de conceptos del dominio, reglas y funcionalidades fuera del alcance.	Equipo	Sección 5.1 y análisis de deuda técnica.
Event Storming	Representación de comandos, eventos, reglas y dudas sobre los procesos principales.	Equipo	Diagrama y reflexión de la sección 5.2.
Historias de usuario	Elaboración de historias y criterios de aceptación para los siete CR.	Equipo	Sección 5.3.
Refactorización	Reorganización del módulo de gestión de productos para separar dominio, aplicación e infraestructura.	Juan Cruz Olguin	Rama refactor/estructura-ddd-gestion-producto y documentación de arquitectura.
CR-001: Validación de datos	Incorporación de validaciones y reglas para rechazar datos inválidos.	Santiago Jesus Mansilla	Rama feature/cr-001, implementación y pruebas.

CR-002: Presentación	Incorporación del catálogo de Presentaciones y su relación con Producto.	Juan Cruz Olguin	Rama feature/CR-002-y-CR-005, implementación y pruebas.
CR-003: SuperLínea	Incorporación de SuperLínea, asociación con Línea, migración de datos e integración en la interfaz.	Elisa Nardi	Rama feature-cr3, migración e implementación.
CR-004: Búsqueda parcial	Búsqueda por denominación, Línea y SuperLínea, combinación de filtros y paginación.	Elisa Nardi	Rama feature-cr4 y pruebas específicas en cr4-test.
CR-005: Denominación automática	Generación de denominación y conservación de la edición manual.	Juan cruz Olguin	Rama feature/CR-002-y-CR-005, implementación y pruebas.
CR-006: Actualización masiva de precios	Aplicación de cambios por porcentaje o monto, con filtros y previsualización.	Agustín Magallanes	Rama feature/CR-006 e implementación documentada.
CR-007: Historial de precios	Registro de valores anteriores y nuevos, motivo, usuario y fecha en la actualización individual.	Trinidad Albarracin	Rama feature/CR-007, implementación y pruebas.
Integración y correcciones	Integración de ramas, resolución de conflictos y ajustes detectados durante las verificaciones.	Equipo	Ramas de integración, merges y commits de corrección.
Testing automatizado	Incorporación y actualización de pruebas por módulos, capas y criterios de aceptación.	Equipo	Sección 5.5, archivos de pruebas y resultados de ejecución.
Documentación y cierre	Consolidación del informe, trazabilidad, métricas, deuda técnica y retrospectiva.	Equipo	Informe final y evidencias adjuntas.

Métricas registradas

Se utilizaron métricas de alcance y resultados de testing. No se calcularon horas de trabajo, productividad por hora ni desvíos temporales, debido a que no se mantuvo un registro de esfuerzo que permitiera obtener esos valores de manera confiable.

Indicador	Valor registrado	Interpretación
Integrantes del equipo	5	Participantes entre los que se distribuyó el trabajo.
Pedidos de cambio contemplados en el informe	7	Alcance documentado: CR-001 a CR-007. No equivale por sí solo a aceptación completa.
Bloques de responsabilidad para testing	5	Distribución por módulos y capas.
Producto y Precios frontend	34 suites y 114 pruebas aprobadas	Resultado de la ejecución informada en la sección 5.5.
Línea y Presentación frontend	13 suites unitarias y 62 pruebas aprobadas; 1 suite E2E adicional implementada	El resultado unitario se distingue de la ejecución de la suite E2E.
Búsqueda por denominación, Línea y SuperLínea	2 suites y 20 pruebas aprobadas	Verificaciones específicas del CR-004 en backend y frontend.

Los valores corresponden a los alcances y ejecuciones documentados. No se presentan como un total global del sistema, ya que existen pruebas compartidas y resultados obtenidos en ejecuciones separadas.

La cantidad de pruebas aprobadas tampoco equivale a un porcentaje de cobertura de código ni garantiza por sí sola el cumplimiento de todos los criterios de aceptación. La cobertura debe interpretarse junto con el alcance medido, los escenarios verificados y las limitaciones de las pruebas.

Gestión de versiones e historial de Git

Git permitió conservar el historial de cambios, desarrollar funcionalidades en ramas y reunir los aportes del equipo mediante integraciones. GitHub se utilizó como repositorio compartido y como evidencia de las versiones publicadas.

Estrategia de branching: El desarrollo se organizó mediante ramas temáticas por cada Change Request (feature/cr-001, feature-cr3, feature-cr4, feature/CR-006, feature/CR-007, feature/CR-002-y-CR-005) y ramas de estabilización de pruebas (release/tests-suite-1), asegurando aislamiento antes de la integración a la rama principal (develop / main).

Evidencia de actividad en repositorio:

<https://github.com/JuanCruzSistemas/proyecto-1/branches/active>

Evidencia de repositorio: <https://github.com/JuanCruzSistemas/proyecto-1.git>

Retrospectiva del equipo

Al cierre de la entrega se revisaron los resultados obtenidos, las dificultades encontradas y las mejoras propuestas para el siguiente ciclo.

Aspecto	Evaluación	Mejora propuesta
Distribución del trabajo	La asignación de CR y módulos permitió avanzar en distintas partes del sistema. Las dependencias exigieron coordinación entre responsables.	Identificar dependencias y criterios de cierre antes de comenzar cada tarea.
Comprensión del código heredado	La estructura y las funcionalidades existentes requirieron un relevamiento previo para comprender el alcance de los cambios.	Mantener un mapa de módulos y una guía de arquitectura actualizada.
Integración	Al reunir las ramas surgieron conflictos y ajustes sobre archivos compartidos.	Integrar cambios más pequeños y revisar periódicamente la compatibilidad entre ramas.
Entorno de ejecución	La configuración de MySQL, migraciones y seeds dificultó algunas verificaciones.	Documentar una secuencia reproducible de instalación y preparación de datos; disponer de una base de testing.

Testing	Las pruebas permitieron verificar reglas de negocio y comportamientos de interfaz. Algunas verificaciones requieren servicios externos o base de datos.	Incorporar pruebas durante el desarrollo y distinguir las ejecutadas con mocks de las que utilizan infraestructura real.
Documentación	El modelo inicial evolucionó durante la implementación y fue necesario revisar su correspondencia con el Event Storming y las historias.	Actualizar la documentación cuando se acuerde una modificación del modelo.
Gestión del trabajo	Git permitió registrar cambios técnicos, pero no ofreció por sí solo una vista completa de tareas, responsables y pendientes.	Complementar el repositorio con un tablero de tareas en GitHub Projects o un tablero Jira/Trello.
Historial de commits	Parte de los mensajes describe claramente el cambio, mientras que otros son demasiado generales.	Acordar una convención común de mensajes y evitar commits identificados únicamente por el nombre de la rama.

La entrega permitió reconocer la importancia de mantener alineados el análisis del dominio, las historias, la implementación y las pruebas. Para la siguiente etapa se propone reforzar el seguimiento de tareas, la integración frecuente y la preparación del entorno de testing.

5.7 Trazabilidad

Pedido de Cambio	Event Storming (Comandos, Reglas y Eventos)	Historia de Usuario	Implementación Backend (Domain / Application / Infrastructure)	Implementación Frontend / UI
CR-001	Regla: Costo > 0, Margen $\in [0,1]$, Precio > 0, Stock ≥ 0, Denominación obligatoria y única Comando: Validar formulario de producto Evento: Formulario validado correctamente / Formulario invalidado.	H.U.1: Rechazo de valores inválidos de costo, margen, precio y stock.	Value Objects: Costo, Margen, Precio, Stock. Exceptions: CostoInvalidoException, MargenInvalidoException, PrecioInvalidoException y StockInvalidoException. DTOs: CreateProductoDto, UpdateProductoDto (class-validator)	Esquemas de validación con Yup en formularios de Producto, bloqueo de valores negativos y mensajes de error unificados.
CR-002	Comando: Agregar presentación. Regla: No eliminar presentación con productos activos. Evento: Presentación asignada / Presentación creada.	H.U.2: Asignación y gestión de variantes de presentación.	Domain: Entidad Presentacion, PoliticaEliminacionPresentacionService. Application: Casos de uso CRUD de presentación. Infra: PresentacionController, relación FK en ProductoOrmEntity	Pantalla de gestión de Presentaciones y selector desplegable integrado en el formulario de productos.

CR-003	Comando: Promover línea a superlínea. Regla: Asociación obligatoria a SuperLínea activa. Evento: Línea promovida a superlínea.	H.U.3.1 / H.U.3.2: Catálogo de SuperLíneas y asociación obligatoria con Línea.	Domain: Entidad SuperLínea, regla de integridad en Línea. Application: Casos de uso de SuperLínea. Infra: Relación Producto → Línea → SuperLínea, migración inicial.	Selector obligatorio de SuperLíneas activas en formulario de Línea, validación Yup y visualización en listados.
CR-004	Lectura (Read Model): Buscar por denominación / línea / superlínea. Filtro: Normalización y combinación de criterios	H.U.4.1 a H.U.4.4: Búsqueda por coincidencia parcial y combinación con paginación.	Application: FindByDenominacionUseCase, FindByProductoUseCase. Infra: ProductoSearchHelper (normalización, ILIKE/LIKE, escape de comodines), PaginacionUtils.	Barra de filtros combinados (Denominación, Línea, SuperLínea), soporte de tecla Enter, botón Limpiar y reinicio de paginación.
CR-005	Regla: Denominación = Marca + Línea + Presentación. Comando: Generar denominación / Editar denominación. Evento: Denominación generada / editada manualmente.	H.U.5: Generación automática y respeto de sobrescritura manual.	Domain: Lógica en Producto.ts con flag denominacionEditadaManualmente. Exception: PresentacionRequeridaException. Application: CreateProductoUseCase.	Campo de texto bidireccional en el formulario: autocompletado en tiempo real al seleccionar relaciones o edición manual protegida

CR-006	Comando: Actualizar precios masivamente. Regla: Precios finales > 0; porcentaje o monto. Evento: Precios actualizados masivamente.	H.U.6: Actualización masiva por porcentaje o monto fijo con previsualización.	Application: AplicarCambioMasivoUseCase, GuardarCambioMasivoUseCase. DTOs: AplicarCambiosMasivosDto. Infra: CambioPreciosController.	Vista previa interactiva de impactos antes de guardar, selector de alcance (global/línea) y validación de montos negativos.
CR-007	Comando: Actualizar historial de precios. Regla: Motivo obligatorio y registro de usuario/fecha. Evento: Historial de precios actualizado	H.U.7: Auditoría y trazabilidad de variaciones de costo, margen y precio.	Domain: Entidad HistorialPrecio, HistorialPrecioFactory. Application: UpdatePrecioUseCase, FindHistorialPrecioUseCase. Infra: Transacción ACID con TypeOrmUnitOfWork.	Modal de modificación de precio individual con vista previa del cálculo, campo de motivo obligatorio y tabla cronológica de auditoría.

Demostración Vertical de Trazabilidad de Extremo a Extremo

1. Fase de Descubrimiento (Event Storming)

Durante la creación del Event Storming se identificó que el cambio de precio no es una simple mutación sobre la tabla de productos, sino una operación de negocio crítica que dispara el comando ActualizarPrecio, exige la regla de cálculo $Precio = Costo \times (1 + Margen)$, valida que el precio resultante sea mayor a cero y genera los eventos PrecioActualizado e Historial de precios actualizado.

2. Fase de Definición (Historias de Usuario y Criterios de Aceptación)

En H.U.7 se formalizó que cada cambio debe exigir un motivo y conservar el estado anterior y nuevo. A su vez, los criterios de aceptación de H.U.1 exigen que el sistema rechace costos negativos y márgenes incoherentes mediante mensajes claros antes de persistir.

3. Fase de Implementación (Arquitectura Limpia en Backend y Frontend):

- **Capa de Presentación / UI:** El usuario interactúa con el componente modal de cambio de precio en React, el cual previsualiza el resultado y envía la petición a través de `producto-service.ts`. En el backend, `CambioPreciosController` intercepta la petición y valida la estructura mediante `UpdatePrecioDto`.
- **Capa de Aplicación:** `UpdatePrecioUseCase` orquesta la transacción. Valida la existencia del producto y del usuario responsable mediante `UsuarioValidator`.
- **Capa de Dominio:** La entidad `Producto` invoca `actualizarDatos(...)`, donde los Value Objects `Costo` y `Margen` validan sus invariantes numéricos y calculan el nuevo `Precio`. Luego, `HistorialPrecioFactory` instancia la entidad de dominio `HistorialPrecio` congelando los valores previos y nuevos.
- **Capa de Infraestructura:** Para cumplir con la atomicidad exigida por el negocio, se utiliza el patrón `TypeOrmUnitOfWork`, ejecutando en una misma transacción de base de datos la actualización del producto (`ProductoRepository.updateEntity`) y la inserción del nuevo registro de auditoría (`HistorialPrecioRepository.save`). Si cualquiera de las dos operaciones falla, se realiza un `rollback` inmediato garantizando la consistencia del inventario.